Project - Low Level Design

on

Haven AI

Course Name: GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Raj Kumawat	EN22CS301779
2.	Rajvardhan Singh Rathore	EN22CS301786
3.	Ronak Bhawsar	EN22CS301824
4.	Rudra Sthapak	EN22CS301828
5.	Rupendra Singh Tomar	EN22CS301833

Group Name: 04D7

Project Number: GAI-27

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3-4
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	4
Chapter 2	System Design	5-9
	2.1. Application Design	5
	2.2. Process Flow	6
	2.3. Information Flow	7
	2.4. Components Design	7
	2.5. Key Design Considerations	8
	2.6. API Catalogue	9
Chapter 3	References	10

1. Introduction

1.1 Scope of the Document

This document provides a comprehensive Low-Level Design (LLD) for the Real Estate Quiz/Tutor Bot, an AI-powered educational system built using Generative AI technologies. The primary focus of this document is to describe the internal structure of the system, including its architectural components, data flow, interaction between modules, and the integration of Large Language Models (LLMs) and vector databases.

The scope includes detailed explanations of how quiz questions are generated dynamically, how user responses are evaluated using semantic understanding, and how personalized feedback is provided to enhance learning outcomes. It also outlines the backend logic, API design, and communication between different layers of the system. Furthermore, the document highlights the use of vector databases for contextual retrieval, ensuring that the AI generates relevant and domain-specific content.

Overall, this document serves as a technical blueprint for developers and evaluators to understand how the system operates at a granular level and how different technologies are integrated to achieve intelligent behavior.

1.2 Intended Audience

This document is intended for software developers, system architects, project evaluators, and technical stakeholders who are involved in the development, review, or assessment of the system. It provides detailed technical insights into the implementation of Generative AI within an educational real estate application.

Additionally, this document is useful for students and researchers who want to understand how modern AI technologies such as LLMs and vector databases can be applied in real-world scenarios. It helps bridge the gap between theoretical concepts and practical implementation by explaining how different components interact within the system. The document is written in a structured and understandable manner so that both technical and semi-technical audiences can benefit from it.

1.3 System Overview

The Real Estate Quiz/Tutor Bot is an interactive and intelligent educational platform designed to enhance users' understanding of real estate concepts through automated assessments. Unlike traditional quiz systems that rely on predefined questions and simple answer matching, this

system uses Generative AI to dynamically generate questions and evaluate responses in a more advanced and meaningful way.

The system leverages Large Language Models (LLMs) to generate quiz questions based on real estate topics such as property listings, pricing, location analysis, and legal aspects. When a user submits an answer, the system evaluates it using semantic analysis rather than exact matching, allowing for more flexible and accurate assessment. In addition to scoring, the system generates detailed explanations for incorrect answers, helping users understand their mistakes and learn effectively.

The local Storage is integrated into the system to store domain-specific knowledge , which are used to provide context to the LLM. This ensures that the generated questions and explanations are relevant and accurate. The backend is implemented using Python and FastAPI, ensuring high performance and scalability. Overall, the system creates a personalized learning loop that continuously improves user knowledge and engagement.

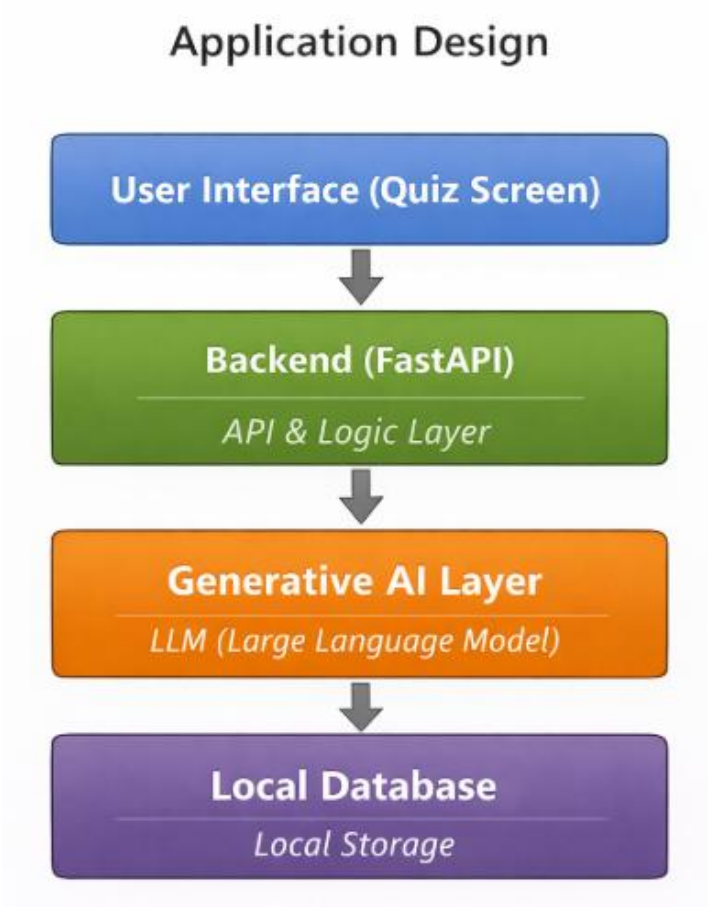
2. System Design

2.1 Application Design

The application is designed using a modular and layered architecture that ensures flexibility, scalability, and ease of maintenance. The system is divided into multiple layers, each responsible for a specific function. The user interface layer handles user interaction, allowing users to start quizzes, answer questions, and view feedback in an intuitive manner.

The backend layer, implemented using Python and FastAPI, acts as the central controller that manages all system operations. It handles API requests, processes user input, and coordinates communication between different components. The Generative AI layer is responsible for generating quiz questions, evaluating answers, and producing explanations using LLM APIs.

This model stores real estate knowledge in the form of local storage , enabling efficient retrieval of relevant information. This layer ensures that the AI model receives proper context before generating responses. The system is designed in a way that each component operates independently but communicates seamlessly with others, making it highly scalable and adaptable to future enhancements.

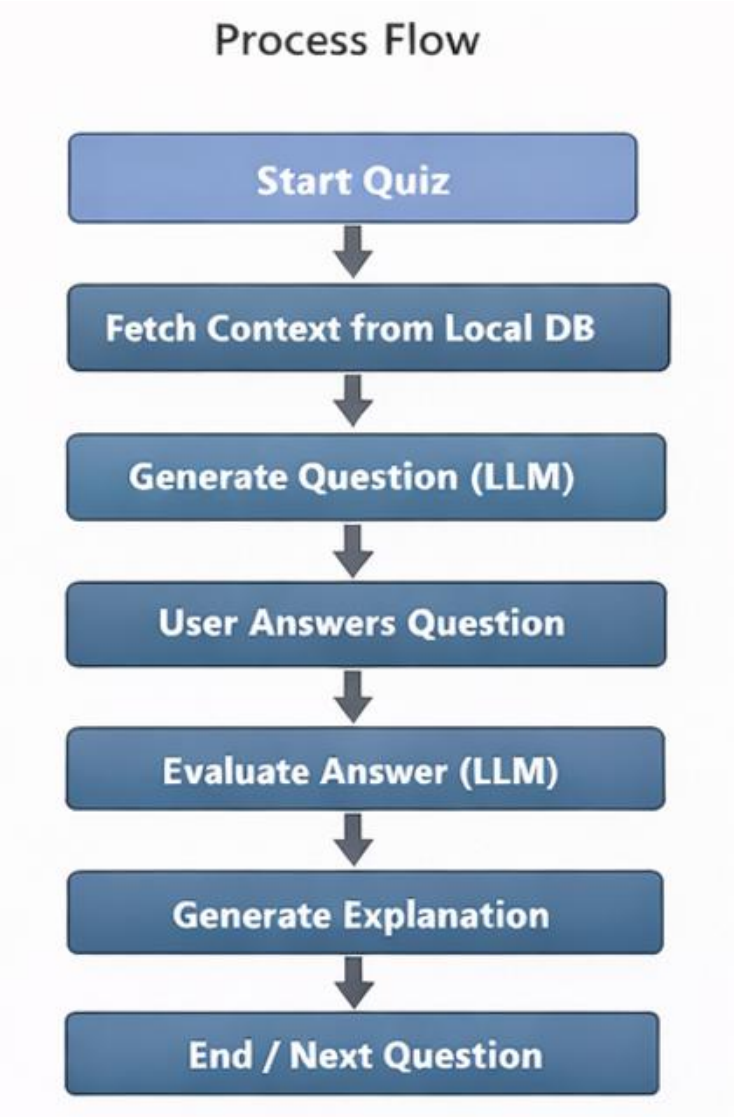


2.2 Process Flow

The process flow of the system describes the sequence of operations that occur during a quiz session. Initially, the user initiates a quiz through the interface, which sends a request to the backend API. The backend then queries the vector database to retrieve relevant real estate information based on the selected topic or context.

This information is passed to the LLM, which generates a quiz question dynamically. The generated question is displayed to the user, who then submits an answer. The backend forwards the user's response along with the original question and context to the LLM for evaluation.

The LLM analyzes the response using semantic understanding and determines its correctness. It assigns a score and generates a detailed explanation, especially if the answer is incorrect. The backend receives this output and formats it into a structured response, which is then sent back to the user interface. This entire process creates a continuous and interactive learning experience for the user.



2.3 Information Flow

The information flow within the system ensures efficient and structured communication between various components. When a user interacts with the system, the input is first received by the backend API. The backend processes this input and sends a request to the local database to retrieve relevant contextual information.

The retrieved data is then passed to the LLM, which uses it to generate a question or evaluate an answer. The LLM produces an output that includes the generated question, evaluation result, and explanation. This output is returned to the backend, which processes it and sends a final response to the user interface.

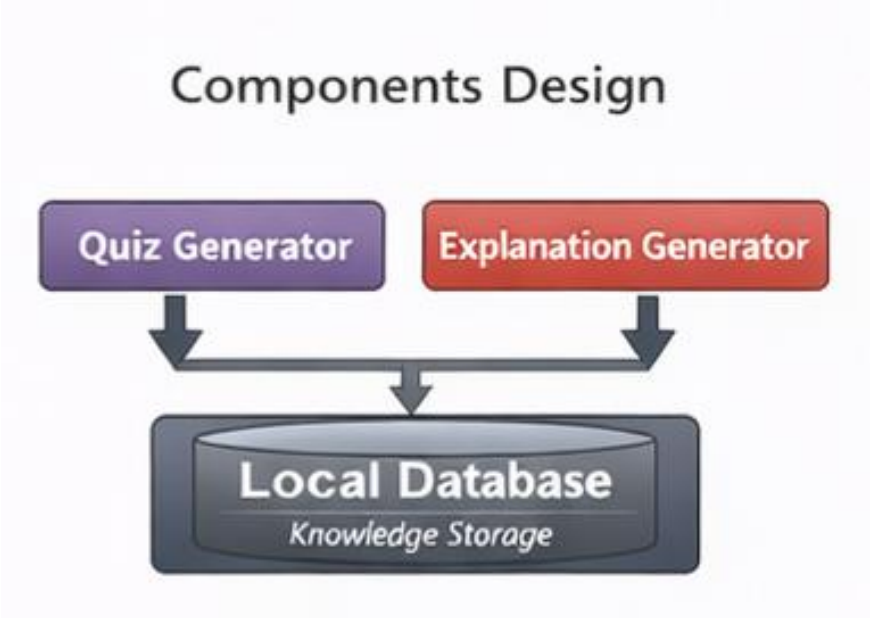
The system maintains a continuous loop of interaction, where user inputs trigger AI-driven processing, and the results are delivered back in real time. This structured flow ensures that the system operates efficiently and provides accurate and meaningful responses to users.

2.4 Components Design

The system is composed of several key components, each designed to perform a specific function. The Quiz Generator component is responsible for creating dynamic quiz questions using the LLM, ensuring that each question is relevant and context-aware. The Answer Evaluation component analyzes user responses using semantic understanding and determines their correctness.

The Explanation Generator component plays a crucial role in enhancing the learning experience by providing detailed feedback and explanations for incorrect answers. The Local Database component stores domain-specific knowledge and embeddings, enabling efficient retrieval of relevant information for the AI model.

The Backend API component acts as the central hub, managing communication between all other components and ensuring smooth execution of system operations. Each component is designed to be modular and independent, allowing for easy updates and scalability.

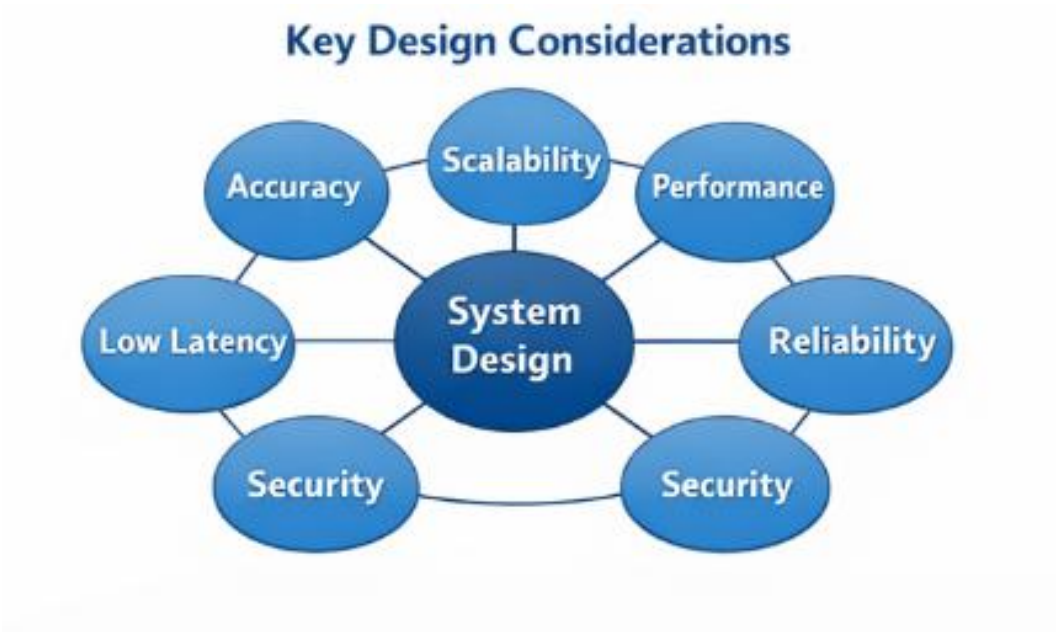


2.5 Key Design Considerations

Several important factors were considered during the design of the system to ensure its effectiveness and reliability. One of the primary considerations is the accuracy of the LLM in generating meaningful questions and evaluating user responses. The system must ensure that the generated content is relevant, correct, and aligned with real estate concepts.

Performance is another critical factor, as the system needs to handle multiple user requests efficiently while maintaining low latency. The use of a Local database improves the quality of generated responses by providing relevant context, which enhances the overall performance of the system.

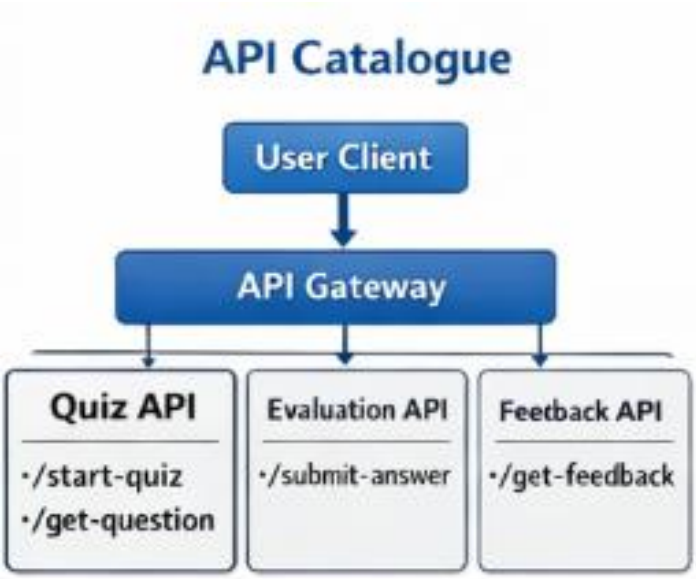
Scalability is also a key consideration, as the system should be able to handle an increasing number of users without performance degradation. The modular design allows for easy expansion and integration of new features. Additionally, the system ensures secure and reliable communication between components, maintaining data integrity and consistency.



2.6 API Catalogue

The system provides a set of RESTful APIs that enable communication between the frontend and backend components. These APIs are designed to handle various operations such as starting a quiz session, retrieving questions, submitting answers, and generating feedback.

The quiz-related APIs allow users to initiate and participate in quiz sessions, while evaluation APIs handle the processing of user responses and generation of scores. Feedback APIs provide detailed explanations and insights to help users improve their understanding. All APIs follow a standardized request-response format using JSON, ensuring compatibility and ease of integration. The APIs are designed to be efficient, scalable, and secure, supporting smooth interaction between different components of the system.



3. References

1. FastAPI Documentation. *FastAPI – Modern, Fast Web Framework for Building APIs with Python.*
   Available at: https://fastapi.tiangolo.com/
2. MongoDB Documentation. *MongoDB Official Documentation for NoSQL Database Management.*
   Available at: https://www.mongodb.com/docs/
3. Docker Inc. *Docker Documentation – Containerization and Deployment Platform.*
   Available at: https://docs.docker.com/
4. LangChain Documentation. *LangChain Framework for Building Applications with Large Language Models.*
   Available at: https://python.langchain.com/
5. OpenAI. *OpenAI API Documentation – Generative AI and LLM Usage.*
   Available at: https://platform.openai.com/docs/
6. FAISS Documentation. *Facebook AI Similarity Search (FAISS) for Vector Database Implementation.*
   Available at: https://faiss.ai/
7. Pinecone Documentation. *Pinecone Vector Database for Semantic Search and Retrieval.*
   Available at: https://www.pinecone.io/docs/
8. ChromaDB Documentation. *Chroma Vector Database for AI Applications.*
   Available at: https://docs.trychroma.com/